WHITE PAPER

AI Might Solve the Junior Developer Problem.

Unfortunately, We Need Junior Developers.

If AI kills the junior role, who becomes senior?

James Storr · October 2026

Contents

Introduction . . . 3

1. The pull request that was quick until review . . . 3

2. Good code is not yet a good change . . . 4

3. The missing repetitions behind judgement . . . 4

4. Standards in the prompt are not standards in the person . . . 6

5. Two pressures on the future talent pipeline . . . 7

6. The argument has serious alternatives . . . 8

7. Plan, understand, then implement . . . 9

8. Embed the workflow around people . . . 11

9. Measure the whole delivery and learning loop . . . 12

10. Who becomes senior? . . . 14

Introduction

We are getting rather good at asking AI to write code. I'm less sure we are giving the people using it enough room to become better engineers.

That matters particularly for juniors. If AI makes the design decisions, writes the implementation and fixes the review comments, a developer can finish the task without working through much of the thinking. The ticket can close before they have properly understood the change.

AI can help with that learning too. I want to explore how we might use it to deliver well while giving people the practice, support and responsibility they need to grow. Some of the time saved at the keyboard could be well spent talking to the person sitting behind it.

1. The pull request that was quick until review

It's interesting to see how quickly engineers have embraced the AI tooling that has arrived over the last two years. It reminds me of my children learning to operate the remote control before they could say twenty words. They soon knew which buttons produced the pictures and sounds they wanted, without understanding what half the other buttons were for. Familiarity with the controls comes remarkably quickly.

A crude comparison, admittedly. In my defence, I'm writing this while watching my youngest navigate Netflix.

In engineering, the results of that familiarity are uneven. Some less experienced engineers are producing fantastic, well-engineered work. Others are producing code that works, but shows little evidence that the decisions behind it have been thought through. This isn't exclusive to junior developers, but it is a pattern in the pull requests (PRs) my colleagues and I review.

Consider a PR produced super fast. The code is readable, the tests pass and the workflow is green. Look closer, though, and the change doesn't follow the established coding standards. Logic sits in the wrong components, responsibilities are muddled, and a future change will require touching code that should never have needed to know about it. The implementation arrived quickly.

The saving is easy to see because it happens first. The bill arrives later, in review and rework, and often lands on someone more experienced. A task that looked cheaper to implement can become more expensive to deliver. Moving the cost onto a senior engineer's calendar does make it rather easier to miss.

The experienced engineers I work with tend to question the proposed approach more. They have years of decisions, mistakes and consequences to draw on, and habits of thinking beyond whether the code runs. Those habits remain useful when AI is doing the typing.

What concerns me is how the junior engineer develops that experience. Working through a problem, making choices and discovering why some of them were poor has always been part of learning the job. If the tool lets someone reach an outcome without working through those decisions, where do they get that practice? Correcting the PR addresses the immediate change. My concern is whether the engineer is also becoming better equipped to make the next one.

Employers have a direct stake in closing that experience gap. When feedback becomes lasting understanding, the engineer can take on more responsibility with less supervision. When the same corrections keep coming back, the business keeps paying for them. More capable AI will change the

decisions engineers need to make, but employers still need people who can apply it well and own the outcome. Developing that capability is part of making the investment in AI pay, and part of building the next generation of senior engineers.

In this paper, I want to explore that concern and what we can do about it: how we keep AI-assisted delivery moving while ensuring the people directing it continue to develop the judgement the work requires.

2. Good code is not yet a good change

Code can compile, pass its tests and do what the ticket asks while still leaving the application harder to maintain. Google's code-review guidance reflects this: reviewers should consider how a change fits the system, alongside whether it works. Google, What to look for in a code review

Take a controller: the code that receives a request and passes it to a service. In this application, permission rules belong in a separate component. AI instead puts those rules inside the controller. The feature works, but a future policy change could mean editing several controllers and keeping their versions in step. The engineer needs to recognise why that placement matters under the application's existing design.

Robert C. Martin's Single Responsibility Principle, discussed in *Clean Architecture*, gives us one way to reason about this. The important question for this paper is whether the developer can recognise the problem and explain the choice. A well-written implementation tells us little about that understanding.

Code review gives us an opportunity to find out. Bacchelli and Bird's research identified understanding the change and sharing knowledge as important parts of review. Bacchelli and Bird, 2013

Ask the developer why the logic belongs where it does, which standard informed the approach and what they questioned or changed before pressing the go-faster button. Their answers can tell us more than another green tick.

We could give AI those standards through a harness: tooling that supplies context and structures the agent's work. That could improve the first attempt. But if the developer accepts the correct implementation without understanding why it is right, we have improved the output while allowing another opportunity to practise judgement to pass by.

3. The missing repetitions behind judgement

Consider how a developer might traditionally have learned from that endpoint. They would read the surrounding code, trace a similar request and decide where the permission logic belonged. An initial attempt might put it in the controller. In review, a colleague could explain the misplaced responsibility and ask them to work through an alternative. Moving the logic would mean understanding the components and checking that the behaviour still held. On the next task, they would have a decision and its consequences to draw on. With useful feedback and repeated practice, something that first needed explaining could become something they recognised themselves. Simply typing the code was never the whole of it.

Today, that review can become a single instruction to AI: “Address the PR comments.” The code can improve without the developer working through why it needed changing.

There is an uncomfortable possibility here: as AI output becomes more convincing, accepting it may become more convenient. The mistakes that once prompted a developer to stop and think may become harder to spot, while the need to question the design remains.

Lee and colleagues’ survey of 319 knowledge workers found that greater confidence in generative AI was associated with less reported critical thinking; greater confidence in their own ability was associated with more. *This self-reported survey does not establish skill loss or a particular effect on junior developers*. It gives us reason to examine how confidence in the tool relates to the scrutiny we apply. Lee et al., 2025

Shen and Tamkin examined learning more directly. In a randomised experiment, 52 programmers worked with an unfamiliar Python library, with or without AI assistance. The AI-assisted group scored lower on an immediate assessment of understanding, code reading and debugging; average time savings were not statistically significant. A descriptive analysis within the AI group linked explanation-seeking and conceptual questioning with stronger learning outcomes; *those interaction styles were not randomly assigned. The sample included experienced programmers. The findings do not establish long-term skill loss or greater harm among juniors*. Shen and Tamkin, 2026

My concern for juniors is that they are still building the judgement we expect them to exercise. An experienced engineer may draw on years of decisions, mistakes and consequences when questioning AI’s approach. A junior has had fewer opportunities to build those reference points. If AI supplies both the initial solution and the response to review, they can get more work finished without necessarily becoming better equipped to judge the next piece. That is the gap I want to understand.

Why the improvement doesn’t always stick

When these issues are raised and coaching follows, I see short-term improvement across experience levels. The engineer checks AI output more closely, questions the choices and pays more attention to the harnesses, agents and skills they use. But the improvement often fades. My suspicion is that it is simply too damn convenient to slip back. A deliberate check takes effort; accepting the next plausible answer is easy. A stronger sign of progress would be seeing that engineer recognise the same principle in a different task and explain how it applies, before a reviewer has to point it out.

The same developer might have struggled without AI. The tool may make it easier to work around a gap it did not create. We should examine that change in behaviour while leaving room for difficulties that were already there.

The temptation to accept an answer is also shaped by expectations around us. A sales pitch that presents AI as a replacement for engineers encourages a different conversation from one about helping them work better. If an employer turns that promise into a delivery expectation, time spent questioning a result can start to look like time the tooling was supposed to have saved.

On YouTube and social media, I see hundreds of videos promising extraordinary results from AI. A claim that you can recreate GTA 6 from a single prompt makes for a better thumbnail than a careful discussion of changing a mature codebase. But an impressive demo tells us little about whether the same approach will work reliably in an existing system. Experience gives you reasons to question that leap. When you are still developing that judgement, a constant stream of confident claims can be persuasive. It is easy to see how a less experienced engineer might get pulled into the hype.

When convincing output meets pressure to ship faster and messages that engineering experience matters less, can we really treat repeated acceptance as an individual failing? The engineer still owns the change. But we should also examine the conditions that make accepting the answer easier to justify than challenging it. Otherwise, we keep riding the same merry-go-round of PR comments and rework.

That leaves a conundrum: how do we preserve useful practice alongside the delivery benefits? Employers have commitments and a bottom line to consider. But if today's speed comes at the expense of tomorrow's capability, the business pays for that too. There has to be a middle ground, right?

4. Standards in the prompt are not standards in the person

When I started exploring how to use AI more effectively in my teams, my main focus was on helping it make better decisions. I worked backwards from the code delivered, through the tickets and technical designs, all the way to the product requirements. The aim was to build context at each stage, giving AI more than an isolated ticket and preserving the connection between the original intent and what we delivered.

I built a harness around spec-driven development. It drew on documentation describing our platform, architecture decisions, standards and development practices.

Alongside that, agentic workflows helped product and engineering teams curate the context specific to each project. They connected requirements, designs and delivery tickets, with the aim of preserving intent from conception through delivery. We also had forums for improving the tooling.

Velocity went up, and the measures we had in place were green. A success! On paper, everything looked rather good.

As time went on, though, I began to see the cracks. The approach worked well when a documented standard covered the decision. It was less dependable when the work moved beyond that guidance.

A new design, unfamiliar requirement or more involved logic could take the work beyond the examples we had documented. AI could still propose a plausible implementation. Sometimes it fitted the system; sometimes it did not. A confident answer did not tell us which one we had.

When it went wrong, meetings followed. Engineers, architects and product managers would work back through the decisions, asking why a particular approach had been taken and why an issue had surfaced so late. "Because AI told me so" would hardly have been a satisfying explanation. The question was what reasoning the engineer could offer beyond the recommendation itself.

Those gaps exposed a limit. More documentation could help, but we also needed engineers who could recognise when the guidance was insufficient and work through the unfamiliar parts. A compliant result alone could not tell us whether the developer understood the decision.

The human side of clarification

You may recognise the intent in approaches such as GitHub's Spec Kit, OpenSpec and AWS's AI-DLC. Their workflows differ, but each provides ways to clarify what should be built before implementation. Asking the engineer to resolve ambiguity is a sensible response to an agent making assumptions on their behalf.

But a workflow can still turn clarification into a barrage of questions. If mental fatigue builds, each new question can start to feel like another hurdle to clear. The urge to choose the first plausible answer and press Enter can overtake the effort to think through its implications. We can end up measuring completion of the interaction while assuming the thinking happened along the way.

Questionnaire research offers a useful caution. Galesic and Bosnjak varied question order in a web survey and found that later questions received quicker responses, shorter written answers and more uniform responses to grouped items. *This is an analogy for sustained engagement, not direct evidence that an engineering harness causes fatigue.* Galesic and Bosnjak, 2009

Engineering decisions can require time to read the code, trace a dependency or compare alternatives. A rapid sequence of prompts, especially with colleagues waiting, can make the interaction feel like a video game: respond, advance, repeat. The workflow needs room to investigate, discuss or admit that an answer is not yet known.

An experienced engineer may have relevant patterns to draw on. Someone still building that experience may need help understanding the options before choosing between them. More time alone does not supply that understanding, and experience does not make anyone immune to accepting a plausible answer.

Buçinca and colleagues tested interaction designs with 199 people choosing food substitutions using simulated AI advice. Approaches encouraging independent consideration, including choosing before seeing the recommendation, reduced acceptance of incorrect advice compared with showing advice and explanations immediately. The most effective designs were also rated less favourably. *This short experiment outside software engineering illustrates a trade-off between ease of use and scrutiny; it does not establish learning effects in a clarification workflow.* Buçinca et al., 2021

We can supply standards, preserve project context and require clarification, yet still know little about the person's understanding. Before placing greater confidence in the outcome, we need to see how the engineer has examined assumptions, handled unfamiliar decisions and explained why the approach fits. Otherwise, passive acceptance simply moves from the generated code to the plan.

For juniors, that means supported opportunities to work through choices, discuss alternatives and apply feedback. We need to judge both the changes delivered and what the engineer becomes able to do on later work. For an employer relying on them to take on greater responsibility, both matter.

5. Two pressures on the future talent pipeline

As leaders, we need to consider two pressures on the future engineering talent pipeline: how many people get the opportunity to enter it, and what they learn before and after they arrive. An organisation's own use of AI is only part of that picture.

Judgement and critical thinking begin developing long before someone's first engineering role. If students routinely use AI to supply the answers, construct the arguments and make the connections for them, they may complete assignments while missing some of the practice those assignments were meant to provide. The cheat code is available before the first day at work. My concern is that a gap in understanding could begin in education and then be carried into a workplace that makes it equally convenient to work around it.

Hiring adds a second pressure. SignalFire's June 2026 talent report put new-graduate and entry-level hiring approximately 65% below 2019 levels among its 12 "Tech Majors", and 76% below 2019 at its

tracked early-stage startups. Yet engineering hiring at those startups was 7% above 2019. Recruitment can recover overall while entry opportunities remain constrained. *These figures come from proprietary profile data covering selected companies and VC-backed startups, not a census of junior software jobs.* SignalFire, 2026

Anthropic offers a more recent, narrower example. In its October 2026 report, the company said the proportion of junior technical hires had fallen from 27% to 9% since early 2025. *Its total technical hiring continued to grow, so the falling share does not by itself establish a fall in the number of juniors hired. It is also one employer with an unusual relationship to AI.* Anthropic, 2026

Neither report establishes how much of the change was caused by AI. But they make the entry route worth examining alongside the learning experience. If fewer people get a first opportunity, and some of those who do receive less practice in exercising judgement, the pressures could compound. Improving one employer's tooling would address only part of that problem.

In the short term, experienced colleagues may absorb the extra review and difficult decisions. That can make the consequences easy to miss. Over time, a narrower intake and weaker development could leave fewer engineers ready to take on wider responsibility. An employer might plan to hire that experience when it needs it, but that depends on someone having developed it elsewhere.

This is a possible future, not an inevitable shortage. AI capabilities will change, roles will evolve and engineers may adapt to these pressures, as engineers do. But that requires conscious effort and foresight. We need to plan for the capability we will need and make room for people to develop it.

6. The argument has serious alternatives

The risks facing junior developers are not inevitable. AI tools and the ways we use them will change, and some of the answers to today's problems may emerge through that development. We should take the possibilities for learning as seriously as the risks of bypassing it.

AI offers an opportunity to develop the very skills I am concerned about losing. A student can ask it to produce an answer, but they can also ask it to challenge their assumptions, explore alternatives and question the reasoning behind their own conclusions. Used this way, AI becomes a learning partner that helps them work through a problem and defend their choices. The same tool that makes it possible to bypass judgement can also help someone practise and strengthen it.

At work, that could mean using AI to trace a request through a codebase, explore why components interact as they do or compare possible improvements. AI never tires of you asking "Why?" Its explanations still need checking against the code and the decisions behind it, but the opportunity to investigate further is valuable. The same tool that can supply a solution can also help an engineer work through it.

Carefully designed AI tutoring can improve learning. Kestin and colleagues studied 194 students in a Harvard undergraduate physics course, comparing a structured AI tutor with active classroom learning across two lessons. Students achieved greater immediate learning gains with the tutor, which used expert-prepared material and teaching guidance. *That supports a particular tutoring approach, without establishing long-term engineering development.* Kestin et al., 2025

Programming evidence also challenges the idea that code generation must undermine learning. In a controlled study of 69 novices aged 10-17, Kazemitabaar and colleagues found improved code-authoring performance with an AI code generator. The overall retention-test difference one week

later was not statistically significant. *That is not proof of equivalence or absence of longer-term harm, but it is a reason to examine the learning environment before assuming an outcome.* Kazemitabaar et al., 2023

Nor did risky reuse of externally supplied code begin with AI. A Stack Overflow answer could be copied into a codebase long before a model could produce the change. Fischer and colleagues' 2017 analysis found security-related Stack Overflow snippets matching published Android code, including snippets classified as insecure. *Static matches cannot tell us what individual developers understood or why they used the code.* They do document a pre-AI concern about reused examples. Fischer et al., 2017

A poorly reasoned PR is therefore not enough to establish that AI caused the gap in judgement. The engineer may lack context, have had little useful coaching or be responding to pressure to finish. Those conditions deserve attention alongside the tool. Otherwise, we risk blaming AI for a development problem that would have needed addressing anyway.

We should also leave room for expertise itself to change. Future roles may place greater value on directing agents, evaluating their work and coordinating changes across a system. Engineers entering the profession with these tools could develop valuable capabilities through a different route from ours. We cannot assume that every skill we acquired will retain the same importance, or that familiarity with AI alone will prepare someone for wider responsibility.

If engineers using AI retain what they learn, apply it to unfamiliar work and take on greater responsibility with less support, that would challenge the concern running through this paper. We should be willing to recognise that outcome. The practical question is how to give it the best chance of happening.

7. Plan, understand, then implement

Less experienced engineers need opportunities to learn, develop judgement and take on responsibility. Organisations need people ready for the work ahead. The practical response I would test is to make that development part of AI-assisted delivery.

A developer who passes a ticket to AI and gets a merged PR has delivered something. We know much less about what they understood. The following workflow could make that reasoning visible. The detail should suit the work: a small, familiar fix may need a few sentences; an unfamiliar design deserves more discussion.

1. **Read and interpret the backlog item.** The developer records a brief explanation, in their own words, of the intended behaviour and their initial approach. This gives them a starting point to examine with AI.
2. **Build the implementation plan with AI.** They use the agent to explore the relevant code and propose an approach, then record or link the plan on the backlog item.
3. **Question and learn from the plan.** They ask AI to explain its proposed choices, unpack unfamiliar concepts and show how the design meets the relevant standards and principles. They explore alternatives and challenge assumptions. Where they cannot resolve a question, they investigate or ask a colleague for help.

4. **Summarise their understanding.** The developer explains why the proposed approach fits, which alternatives they considered, where the logic belongs and how relevant coding standards and SOLID principles inform the design. Naming the principles is only useful if they can explain why they apply here.
5. **Use AI to implement the agreed approach.** The plan and its reasoning provide context for the work.
6. **Review the delivered change.** The developer checks it against the requirements and plan, records how they verified the behaviour and explains any departures from the proposed design.
7. **Discuss the change in PR review.** A colleague reviews the implementation alongside the plan and the developer's explanations. The review becomes a conversation about the choices and their consequences, as well as the code itself.

AI can help accelerate the engineer's understanding as well as the implementation. They can ask for an explanation at a level that makes sense to them, work through an example and explore how a principle applies to the code in front of them. They can keep asking without worrying about being judged or wearing out someone's patience, and ask AI to adapt its language and examples to suit them. We should encourage engineers to develop the skills to use AI as a source of learning: asking useful follow-up questions, pursuing what remains unclear and checking explanations against the code and standards.

The shortcut is obvious: ask AI to write the summary of understanding, paste it into the ticket and tick the box. We have another completed field, but little assurance that the developer understands the approach. AI can help them learn; its explanation alone cannot demonstrate that they have.

This is where leaders need to use their people skills and invest some time. A ten-minute conversation, asking the developer to explain their choices, discuss alternatives and respond to follow-up questions, could tell us far more than checking whether a summary exists. It also creates an opportunity to explore uncertainty and help them work through what they have not yet understood.

Suppose AI puts permission logic in the controller, in both its plan and its code. Checking one against the other would miss the design problem. Ask instead: if permissions changed but the endpoint's contract stayed the same, what would need editing? Needing to change the controller is a cue to examine its responsibilities against the application's conventions and an established endpoint.

A junior may first need a reviewer to point this out. They can use AI to compare the two designs and trace what a future rule change would affect, then explain their understanding back to the reviewer. On a later endpoint, that experience gives them a reference point for questioning AI's proposal. Repeated practice and feedback could turn a reviewer's observation into a habit of their own. We need to check whether that progress happens.

Sounds like more work, right? Some of it is. Questioning, explaining and reviewing take time. Where AI reduces implementation time, I would invest some of that saving in understanding. Whether it pays back through less rework and greater independence needs testing.

Keep the plan, reasoning and feedback on the backlog item so they can be revisited. The useful sign of progress is an engineer applying the lesson on later work, rather than a growing collection of completed fields.

If we reward only the speed of the immediate delivery, we give people little reason to protect time for this work. Development can remain something we value on a poster or in an HR objectives list, while

the daily workflow rewards getting the next ticket closed. Leaders need to make learning part of what successful delivery means.

The controller example shows this at the level of a single change. Engineering judgement also shapes the requirements, designs and standards that guide the work. We should look at who gets to participate in those decisions too.

8. Embed the workflow around people

Embedding this workflow around people means giving them a meaningful part in shaping the work, with colleagues who can challenge and guide them. Leaders need to make room for that in everyday delivery.

Some of the learning starts before an implementation plan exists: working through a requirement, questioning an architectural decision or developing a standard. The context we supply to AI is itself engineering work that people can help shape.

To explore where that involvement can happen, I find it useful to group the information guiding delivery into two categories: strategic context and project context.

Strategic context carries across projects: architecture, coding standards, design principles, constraints and the reasons behind earlier decisions. It may live in an engineering strategy, architecture decision records, scattered documents or tribal knowledge. It explains the boundaries people work within and why they exist.

Project context concerns a particular initiative or change: the problem, who it matters to, the intended outcome and the requirements, constraints and assumptions. It also records the proposed design, alternatives and reasons for the choices made. It connects the platform's enduring guidance to the decisions this work requires.

Backlog tickets, scope documents, technical designs and implementation plans should provide a connected account of that intent. The context should develop as understanding improves, keeping the reasoning visible through implementation and review.

The question for leaders is who gets to help shape each type of context, and how that involvement can build their understanding.

Architects and lead engineers often take responsibility for shaping the strategic context, drawing on their experience to establish the guidance others will use. I have found it beneficial to involve less experienced developers in that work too. Working through why a standard exists, what alternatives it excludes and where it might not apply gives them something to reason about when they later use it.

Think about the difference between following a decision and helping work through it. In the second case, you have considered the options and their consequences. That is the involvement I want to create, with guidance suited to the engineer's experience.

Research on choice offers some support. Patall and colleagues' meta-analysis of 41 studies found modest average improvements in intrinsic motivation and effort when people were offered choice. *Choice is not ownership, and those results do not establish better engineering judgement or lasting learning*. They do give us reason to consider how much influence people have over their work. Patall et al., 2008

The same thinking applies to project context. Consider a team working through an initiative with an AI harness. There may be opportunities to review every document it produces, yet a junior engineer could still be a passenger: someone else drives the discussion, resolves the questions and takes responsibility for the design. The junior is present, but has little occasion to form or defend a view. This problem predates AI; dividing up manually written code would not, by itself, resolve it.

An alternative is to give that engineer responsibility for a bounded part of the initiative, with support suited to their experience. They could investigate a design question, use AI to explore options and bring a recommendation back to the team. They would explain the alternatives, how the proposal fits the standards and principles, and how it serves the business need. A colleague can challenge the reasoning and help them work through gaps in their understanding.

Where the scope is suitable, that responsibility could continue through delivery of a complete feature, giving the engineer a chance to see how their earlier decisions affect the implementation, review and outcome. Being the hands at the terminal is only part of it. They need a meaningful part in the decisions and an opportunity to follow their consequences.

AI can perform more of the implementation while the engineer remains actively involved in shaping the work. That is where we can preserve agency and create opportunities to develop the judgement we will need them to exercise in future.

9. Measure the whole delivery and learning loop

Giving engineers more time to think, learn and talk through their decisions has a cost. We should be able to explain what we are getting for it. Otherwise, that time risks being the first thing dropped when the next deadline arrives.

Features delivered and money earned still matter. But a healthy delivery report tells us little about whether someone is becoming a better engineer. For that, we need to look at the work more closely: the mistakes they stop repeating, the decisions they can explain and the problems they can take on with less guidance.

We have some useful guidance already. The SPACE framework warns against trying to capture developer productivity in a single number, such as code volume. DevEx recommends looking at both what happens in the workflow and how developers experience it. *Neither proves this proposal will work, but both support looking beyond the busiest person on the dashboard.* Forsgren et al., 2021; Noda et al., 2023

Count the whole job

Consider two similar changes, both delivered using AI. One moves quickly into implementation and needs substantial correction in review. The other includes time to plan and understand the approach first. The figures below are made up to illustrate the comparison; they are not measured results or a promise of savings.

Where people spend their time	Change needing substantial correction	Change with more planning and learning
Planning and learning	0 minutes	30 minutes
AI-assisted implementation	30 minutes	30 minutes
Developer checks	15 minutes	20 minutes
Colleague's review	60 minutes	20 minutes
Developer corrections	75 minutes	10 minutes
Total human effort	**180 minutes**	**110 minutes**

The extra thirty minutes at the beginning is easy to spot. So is the hour a colleague spends reviewing the first change, if we bother to count it. The comparison needs to include everyone's time before we decide whether the learning is an expense we cannot afford.

Of course, the real figures could come out differently. The work needs to be similar enough to compare, and the quality of the result still matters. Fewer review comments might mean a better change, or a less thorough review. We need to read the work as well as count it.

These figures also describe time spent working. A PR waiting two days for review is a different problem from one that takes two days to review. Both matter, but they tell us different things.

Look at the next task

Our controller example gives us somewhere concrete to start. On a later endpoint, does the engineer spot permission logic in the wrong place before a reviewer points it out? Can they explain why it belongs elsewhere under the application's conventions? That would give us something more useful than another completed training objective.

Then look beyond the familiar example. Suppose the engineer is working on a background job and questions where its permission policy should live. They may still need help with the answer, but recognising the decision and investigating it would suggest they are beginning to apply the principle more widely.

A short conversation can explore that understanding. Ask what would happen if permissions became specific to each customer. Which part of the design would change? What would they need to check? We are looking for someone who can work through the consequences, including admitting where they are unsure.

Keep a brief note of examples like these and discuss them with the engineer. Useful peer feedback might describe how they questioned a design, investigated an alternative or caught a problem before implementation. “Doing well” is encouraging, but it gives us very little to work with.

The same applies to wider responsibility. An engineer who researches a design question, brings back a recommendation and follows it through delivery is giving us more to assess than the number of tickets they close. Asking for help along the way should not count against them. Knowing when to ask is part of the job.

Include the time people spend helping

We could also track whether the same corrections keep returning. Imagine that four out of six comparable PRs initially needed the same architectural correction, followed by one of the next six. That would be worth discussing. It would not settle the matter: perhaps the tasks became easier, the reviewer changed or someone else supplied the decisions.

Look at the support behind those changes too. A shorter review may follow a longer design conversation. That could be a good investment, especially if the engineer is taking on harder work, but we should include that time honestly. Moving the effort to another meeting does not make it disappear.

Start small and check what changes

I would start with a few recent changes to understand where the effort currently goes. Then follow similar work using the proposed approach. Agree with the engineer what they are trying to get better at, keep a few examples of their decisions and revisit them together. Ask where AI helped them learn, where they still needed support and whether the workflow gave them room to think.

If the same mistakes become less frequent, the reasoning improves and the engineer can take on more responsibility, we have reasons to keep going. If we get beautifully completed tickets but keep revisiting the same gaps in understanding, we need to change the approach.

A short trial can help us see whether the workflow is practical and whether progress is beginning. Understanding how well that learning lasts will take longer. We should be able to point to the decisions the engineer can now make, the support they still need and what we will help them tackle next.

10. Who becomes senior?

The pull request that arrived quickly and needed rebuilding in review gives us an immediate reason to pay attention. The time saved generating code can disappear into someone else’s review and rework. But if the developer leaves that process little better equipped to judge the next change, the cost could extend well beyond the current ticket.

AI can also help close that gap. It can explain an unfamiliar concept, help explore alternatives and give an engineer room to keep asking questions. The evidence we have examined leaves room for both better learning and missed learning. We should remain open to different routes into expertise as the work changes. We should also be deliberate about giving people the practice they need along the way.

The workflow proposed here is a starting point to test. Work through a plan with AI, use it to understand and challenge the choices, then explain the approach before implementation. Follow that reasoning through the delivered change and its review. Give less experienced engineers a meaningful part in shaping requirements, designs and standards, with colleagues who help them take on more responsibility.

That asks something of leaders too. A summary of understanding is remarkably easy to acquire when the same tool can write the plan, the code and the summary. Finding out what someone has learned takes a conversation. Helping them work through what they have not understood takes patience. These are familiar parts of developing engineers, and they deserve a place in how we adopt AI.

There is a business case to examine here, alongside the responsibility to develop people. Repeated corrections cost time. So does coaching. We need to account for both and look for progress in the work that follows: better decisions, fewer repeated mistakes and greater independence. If the proposed approach mainly produces more documentation, we should change it. The investment needs to show up in what engineers become able to do.

The concern behind the title is therefore worth taking seriously, even though the future shape of the junior role remains uncertain. An organisation that expects to hire experienced engineers later is relying on someone having helped them become experienced. Its own juniors are part of that future supply.

When AI saves us time on implementation, we have a choice about where that time goes. Some of it should go into helping a less experienced engineer understand the system, question a design and take responsibility for a decision. That is an investment in the people we will eventually expect to guide the work and develop the engineers who follow them.